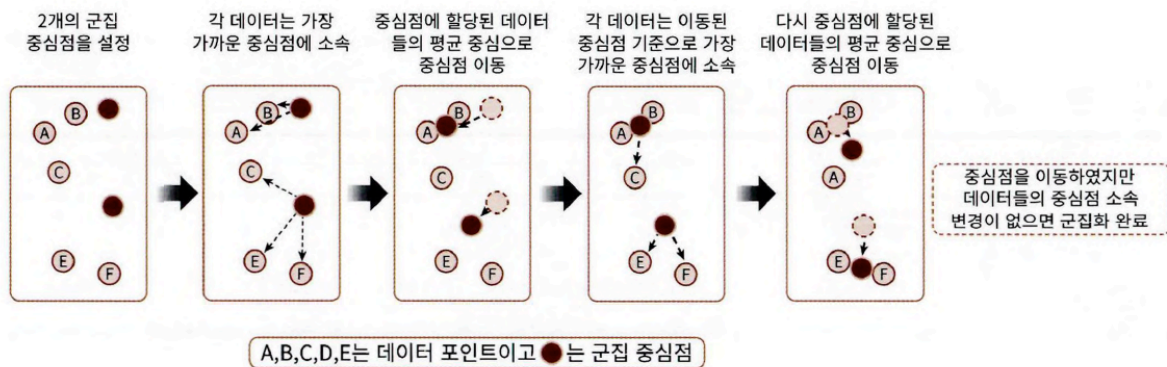


07. 군집화

📌 01 K-평균(K-Means) 알고리즘 개요

✅ 작동 방식

1. 중심점 초기화: 군집 수(K)만큼 임의의 위치에 중심점 배치
2. 데이터 할당: 각 데이터를 가장 가까운 중심점에 할당
3. 중심점 재계산: 할당된 데이터들의 평균 좌표로 중심점 이동
4. 반복 수행: 중심점 이동 및 재할당을 반복
5. 종료 조건: 더 이상 중심점 이동이 없을 때 종료



✅ 장점

- 구현이 간단하고 빠름
- 다양한 군집화 문제에 널리 사용됨

✅ 단점

- 거리 기반 → 고차원 데이터에 부정확
- 군집 수(K)를 사전에 알아야 함
- 반복 수행 → 계산 시간이 오래 걸릴 수 있음

✅ KMeans 클래스 (사이킷런)

- `n_clusters`: 군집 수 (중요)
- `init`: 초기 중심점 설정 방식 (기본값: k-means++)

- `max_iter` : 최대 반복 횟수
- 주요 속성: `labels_` (각 데이터의 군집 라벨), `cluster_centers_` (중심점 좌표)

✓ 붓꽃 데이터셋 군집화 실습

- 꽃받침(sepal)과 꽃잎(petal)의 길이/너비를 기준으로 군집화
- `n_clusters = 3` 으로 설정하여 군집화 수행
- 실제 분류(target)과 비교 시 일부 군집이 잘 나뉘고, 일부는 혼합됨
- PCA로 4차원 데이터를 2차원으로 줄여 시각화

📌 군집화 테스트용 데이터 생성

✓ `make_blobs()`

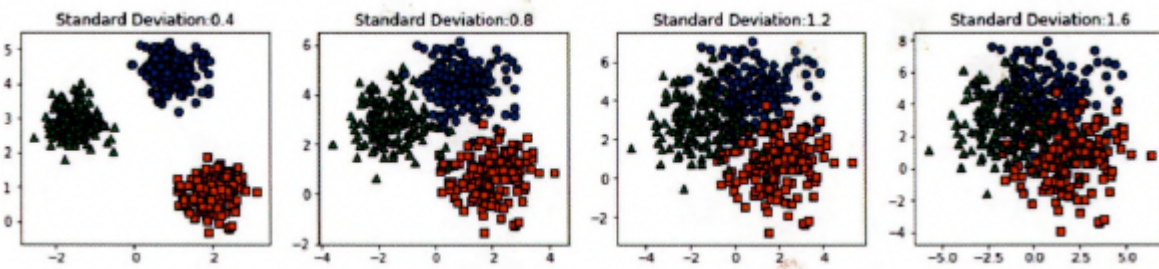
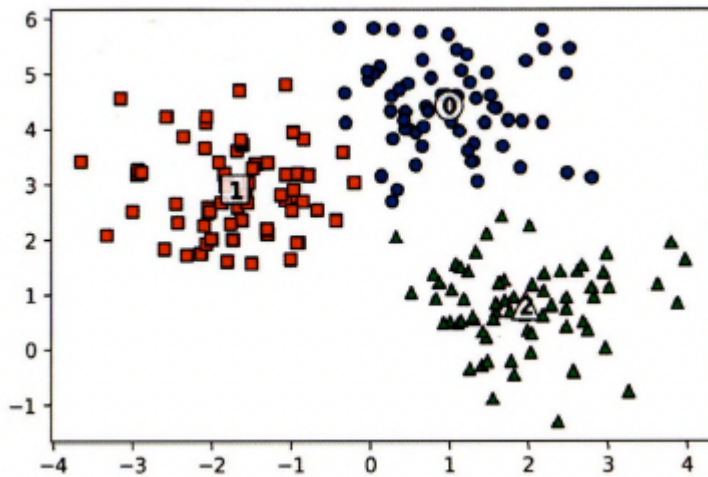
- 용도: 군집화용 테스트 데이터 생성
- 주요 파라미터
 - `n_samples` : 샘플 수
 - `n_features` : 피쳐 수
 - `centers` : 군집 수 또는 중심 좌표
 - `cluster_std` : 군집 내 데이터 분산 정도

✓ `make_classification()` / `make_circles()` / `make_moons()`

- 분류 및 복잡한 형태의 군집화 테스트용 데이터 생성 가능

📌 실습 요약

- `make_blobs()` 로 생성한 데이터를 KMeans로 군집화
- `fit_predict()` 로 군집화 수행 후 `cluster_centers_` 로 중심 확인
- 군집화 결과는 `target` 과 1:1로 정확히 대응되지 않을 수 있음
- `cluster_std` 값에 따라 군집이 더 조밀하거나 퍼짐



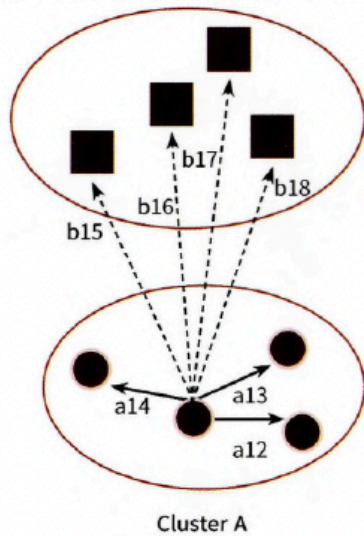
📌 02 군집 평가(Cluster Evaluation)

✅ 군집 평가의 필요성

- 대부분의 군집화 데이터는 비교할 타겟 레이블이 없어 평가가 어렵습니다.
- 군집화는 분류와 다르게 데이터 내 숨겨진 그룹을 찾는 데 목적이 있습니다.
- 따라서 군집화의 성능을 평가하기 위한 방법이 필요합니다.

✅ 실루엣 분석(Silhouette Analysis)

Cluster B
(Cluster A의 1번 데이터에서 가장 가까운 타 클러스터)



Cluster C

- a_{ij} 는 i 번째 데이터에서 자신이 속한 클러스터내의 다른 데이터 포인트까지의 거리. 즉 a_{12} 는 1번 데이터에서 2번 데이터까지의 거리
- $a(i)$ 는 i 번째 데이터에서 자신이 속한 클러스터내의 다른 데이터 포인트들의 평균 거리. 즉 $a(i) = \text{평균}(a_{12}, a_{13}, a_{14})$
- $b(i)$ 는 i 번째 데이터에서 가장 가까운 타 클러스터내의 다른 데이터 포인트들의 평균 거리. 즉 $b(i) = \text{평균}(b_{15}, b_{16}, b_{17}, b_{18})$

- 정의: 각 군집 간 거리 분리가 얼마나 효율적인지 판단.
- 기준:
 - 동일 군집 내 데이터는 서로 가깝게
 - 서로 다른 군집 간 거리는 멀게

✓ 실루엣 계수(Silhouette Coefficient)

- 정의: 개별 데이터가 얼마나 잘 군집화됐는지를 측정
- 계산 방식:
 - $a(i)$: 같은 군집 내 다른 포인트들과의 평균 거리
 - $b(i)$: 가장 가까운 타 군집과의 평균 거리
 - $s(i) = (b(i) - a(i)) / \max(a(i), b(i))$
- 값의 범위:
 - 1에 가까울수록 잘 군집화됨
 - 0에 가까우면 경계에 위치
 - 음수면 잘못된 군집화 가능성

✓ 사이킷런 실루엣 관련 함수

- `silhouette_samples(X, labels)`: 각 데이터 포인트의 실루엣 계수 반환
- `silhouette_score(X, labels)`: 전체 평균 실루엣 계수 반환

✓ 좋은 군집화 조건

1. 전체 실루엣 계수 평균값이 1에 가까울수록 좋음
2. 각 군집의 실루엣 계수 평균값의 편차가 작아야 함

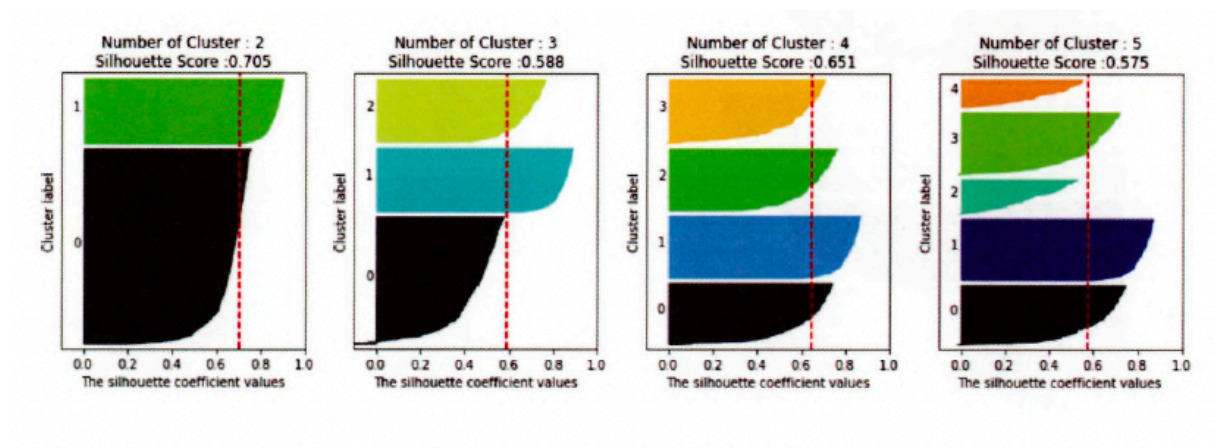
✓ 붓꽃 데이터 예시

- 평균 실루엣 계수 ≈ 0.553
- 1번 군집: 평균 실루엣 계수 약 0.79
- 0번/2번 군집: 각각 약 0.41, 0.45로 낮음

✓ 군집 수 최적화 시각화

- 단순히 평균 실루엣 계수만 높다고 최적 군집 수는 아님
- 각 군집별 실루엣 계수 분포를 시각화하여 적절한 군집 수 판단

실루엣 도표 예시



- 군집 수 = 2:
 - 평균 계수 ≈ 0.704 (높음)
 - 하지만 일부 군집의 내부 거리가 멀거나 인접 군집과 가까움
- 군집 수 = 3:
 - 평균 계수 ≈ 0.588
 - 일부 군집은 평균보다 낮은 계수 $\rightarrow$ 적절하지 않을 수 있음
- 군집 수 = 4:
 - 평균 계수 ≈ 0.65

- 모든 군집이 비교적 균일하게 구성되어 가장 적절한 군집 수로 판단됨

✓ 시각화 도구 예시

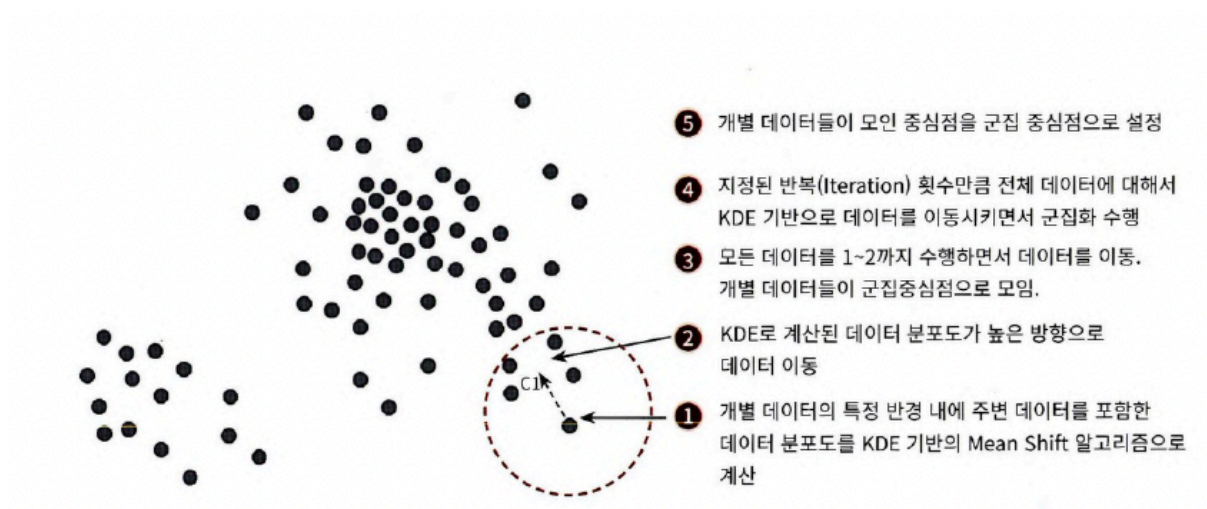
- `visualize_silhouette(cluster_list, X_features)` :
 - 다양한 군집 수에 따른 실루엣 계수 시각화
 - `make_blobs()`로 생성한 테스트 데이터에 활용 가능

📌 03 평균 이동(Mean Shift)의 개요

✓ Mean Shift란?

- K-평균과 유사하지만, 중심을 데이터 밀도가 가장 높은 지점으로 이동함.
- 확률 밀도 함수(PDF)를 이용하여 군집 중심점을 찾음.
- 주로 KDE(Kernel Density Estimation) 를 통해 데이터가 가장 밀집된 위치 탐색.

✓ 알고리즘 작동 방식

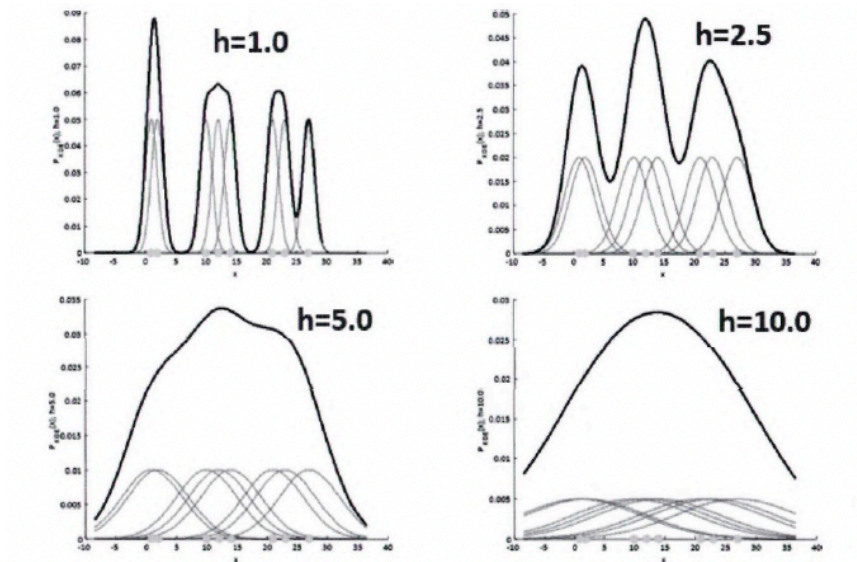


1. 초기 중심점에서 주변 데이터를 바탕으로 KDE 계산.
2. KDE 값이 높은 방향으로 중심점 이동.
3. 전체 데이터에 대해 위 과정을 반복 → 중심점으로 데이터가 수렴 → 군집 형성.

✓ KDE (Kernel Density Estimation)

- 커널 함수를 이용해 확률 밀도 함수를 추정.
- 대표적인 커널: 가우시안 커널

- KDE = 개별 관측값에 커널 적용 → 합산 → 관측 개수로 나눔.
- KDE의 대역폭(h) 설정은 매우 중요:
 - h 작을수록 → 뾰족한 밀도 곡선 (과적합 우려)
 - h 클수록 → 부드러운 밀도 곡선 (과소적합 우려)

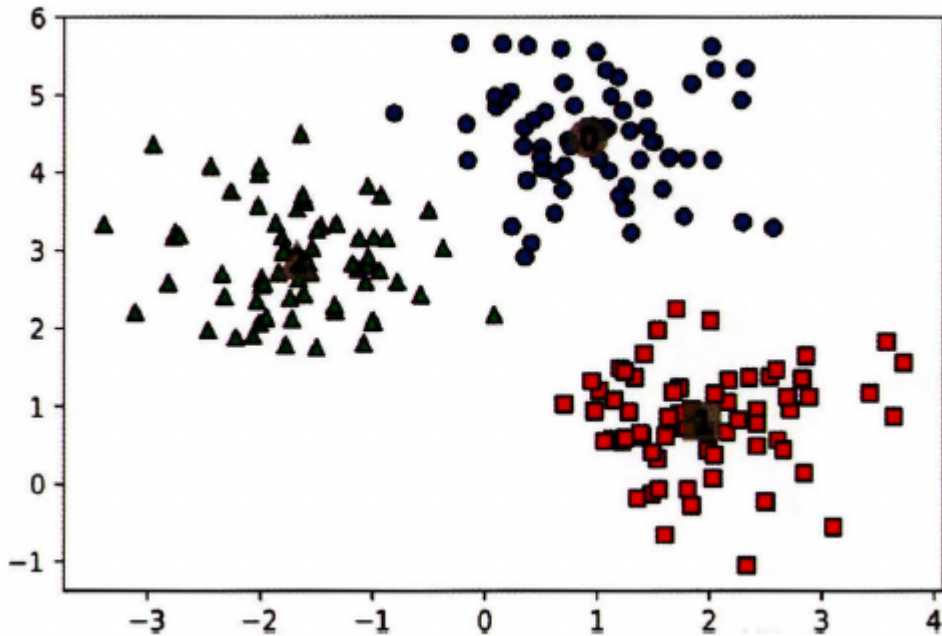


✓ Mean Shift의 핵심 파라미터: **bandwidth**

- KDE의 대역폭과 같으며 군집화 품질에 매우 큰 영향을 줌.
- bandwidth ↓ → 군집 수 ↑ (세분화)
- bandwidth ↑ → 군집 수 ↓ (일반화)
- bandwidth 자동 계산 함수: `sklearn.cluster.estimate_bandwidth()`
- `make_blobs()` 로 생성한 데이터 사용.
- bandwidth=0.8 → 6개 군집 (세분화)
- bandwidth=1.0 → 3개 군집 (적절한 군집화)

✓ MeanShift 클래스 (사이킷런)

- 주요 속성:
 - `cluster_centers_` : 군집 중심 좌표
 - `labels_` : 데이터의 군집 소속
- 군집 수를 미리 정할 필요 없음
- 이상치에 민감하지 않음



✓ Mean Shift의 장점

- 데이터 분포를 유연하게 반영
- 군집 개수를 지정하지 않아도 됨
- 이상치 영향 적음

✓ Mean Shift의 단점

- 수행 시간 오래 걸림
- bandwidth 설정에 민감

✓ 활용 분야

- 일반 데이터 분석보다는 컴퓨터 비전 분야에 적합:
 - 객체 식별
 - 움직임 추적

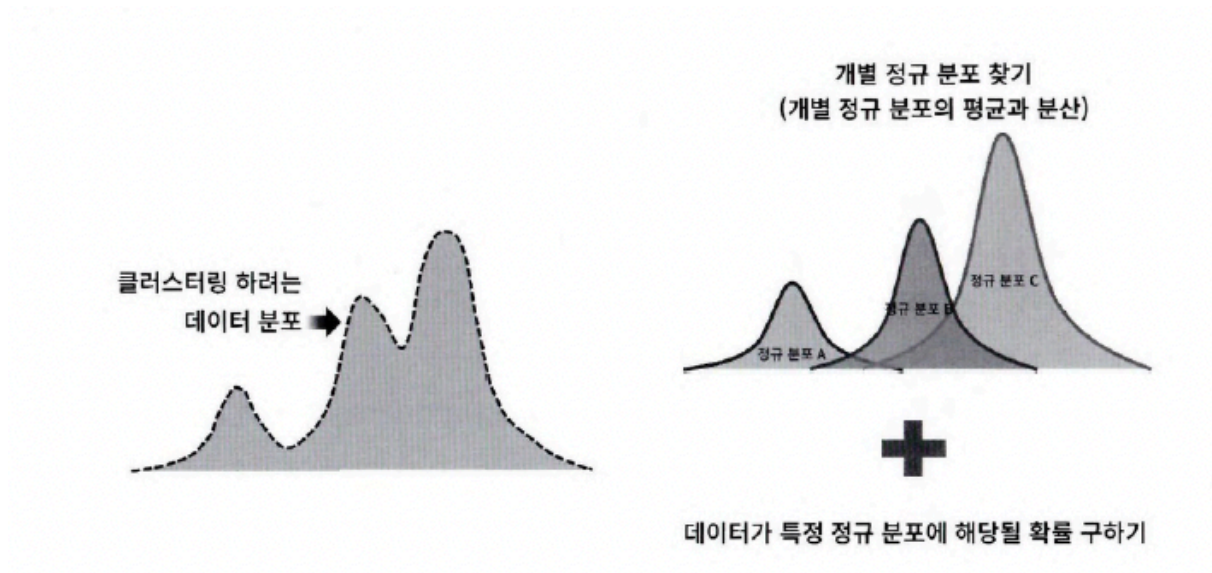
📌 04. GMM (Gaussian Mixture Model)

✓ GMM 개요

GMM은 데이터가 **여러 개의 가우시안 분포(정규 분포)**로 섞여 생성되었다는 가정 하에 군집화를 수행하는 방법입니다.

- 정규 분포는 평균을 중심으로 좌우 대칭적인 종(Bell) 모양을 가진 연속 확률 분포입니다.
- GMM은 이러한 정규 분포들이 섞여서 만들어진 복합 분포라고 보고, 각 분포를 찾아내고 군집화하는 것을 목표로 합니다.

✓ 핵심 개념



- GMM은 각 데이터를 어떤 가우시안 분포에 속할 확률로 평가합니다.
- 두 가지 중요한 모수(파라미터)를 추정합니다:
 1. 각 정규 분포의 평균(μ), 분산(σ^2)
 2. 각 데이터가 어떤 정규 분포에 속할 확률

이를 위해 GMM은 EM 알고리즘(Expectation-Maximization)을 사용하여 반복적으로 모수를 최적화합니다.

✓ 사이킷런의 GMM 클래스

- `sklearn.mixture.GaussianMixture` 클래스를 사용합니다.
- 주요 파라미터는 `n_components`: 군집(정규 분포)의 개수입니다.
- 사용 방법은 KMeans와 유사하며 `fit()`, `predict()` 메서드를 사용해 군집 레이블을 예측합니다.

✓ 예제: 붓꽃 데이터에 GMM 적용

- `n_components=3`으로 GMM 수행

- 결과:
 - Target 0 → cluster 0 (완벽 매핑)
 - Target 2 → cluster 1 (완벽 매핑)
 - Target 1 → cluster 2에 대부분 매핑 (90%), 일부 cluster 1 (10%)

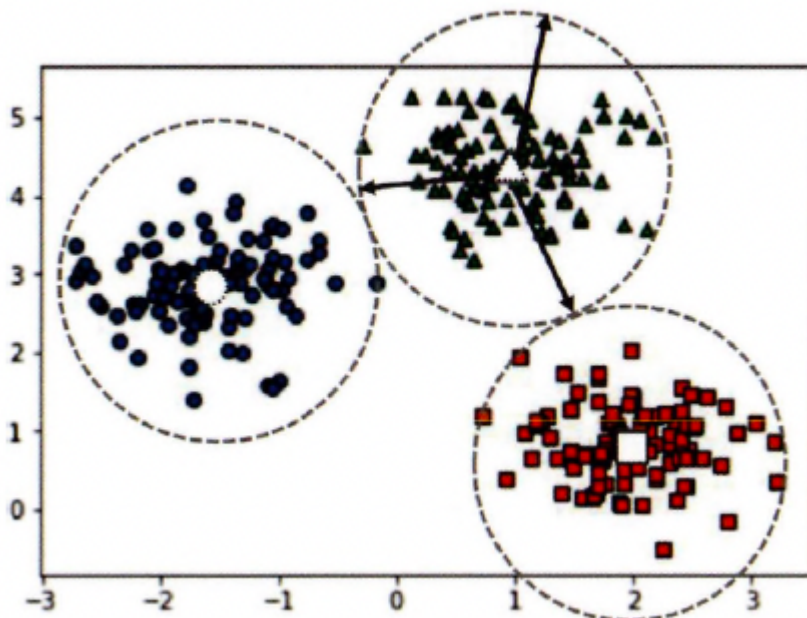
→ KMeans보다 더 정확한 군집화 결과가 나옴

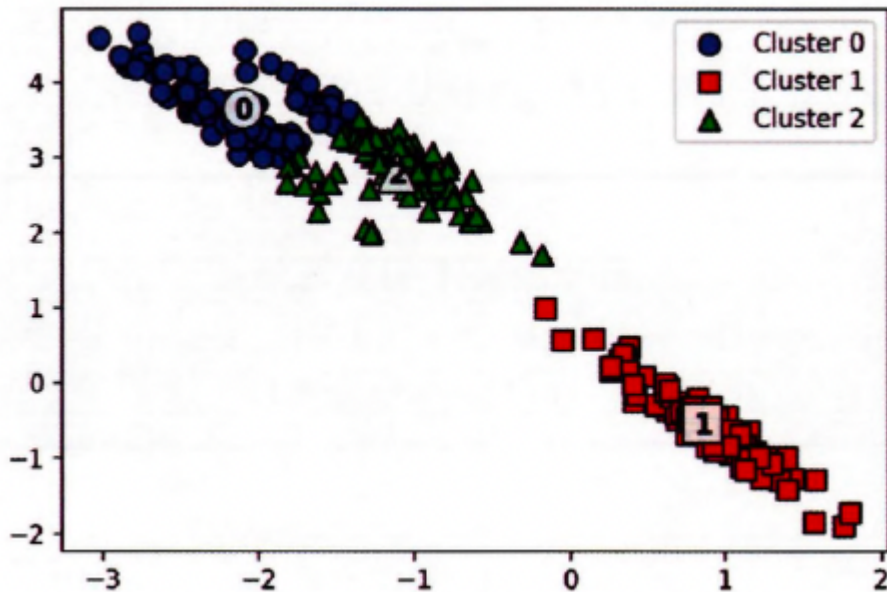
(단, 이는 GMM이 더 우수하다기보다는 데이터 특성에 따라 GMM이 더 적합했기 때문임)

✅ GMM vs KMeans

구분	KMeans	GMM
군집화 방식	거리 기반	확률 기반
군집 형태	원형(Radial) 중심	타원형도 가능 (분포 방향 고려)
군집 중심	계산 가능 (<code>cluster_centers_</code>)	없음
속도	빠름	느림 (EM 반복 학습 때문)
유연성	낮음	높음 (다양한 형태 지원)

Kmeans는 원형의 범위를 가지고 Clustering을 수행





예시 비교

- 원형 분포 (cluster_std 작음): KMeans 우수
- 타원형 분포 또는 방향성 있는 데이터: GMM이 더 우수

✓ 실습용 시각화 함수 (visualize_cluster_plot)

- 사이킷런의 군집 객체(KMeans, GMM) 결과 시각화 가능
- 군집 중심 시각화 여부(`iscenter`)도 제어 가능

✓ 정리

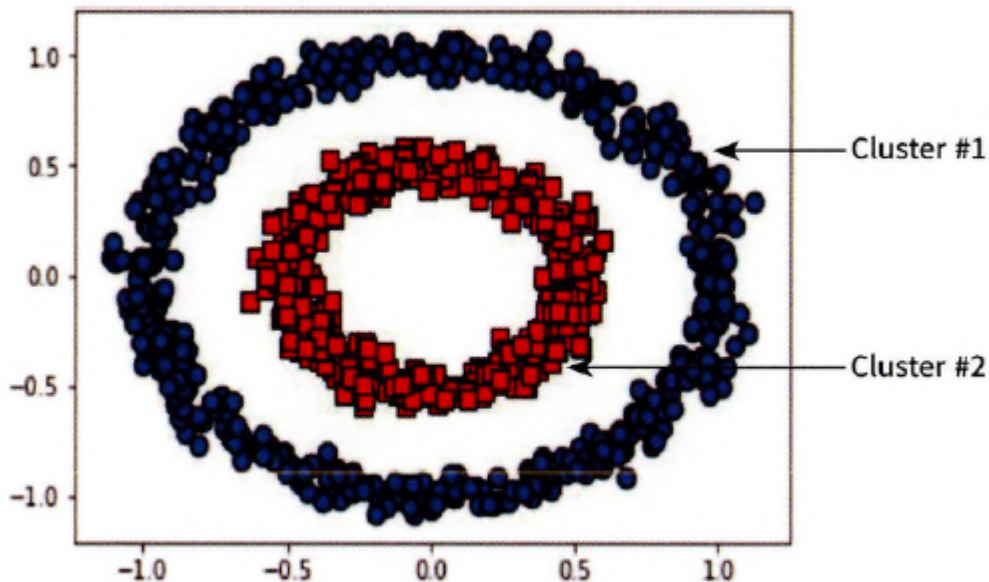
- GMM은 데이터가 여러 개의 정규 분포로 구성되었음을 가정하고, 확률 기반으로 군집화를 수행
- 데이터 분포가 비원형이거나 복잡할 때 KMeans보다 효과적
- 다만 수행 시간이 오래 걸림 (EM 알고리즘 반복 때문)

📌 05. DBSCAN

✓ DBSCAN(Density Based Spatial Clustering of Applications with Noise) 개요

- DBSCAN은 밀도 기반 군집화 알고리즘으로, 데이터의 분포가 기하학적으로 복잡해도 효과적으로 군집화를 수행할 수 있습니다.

- K-평균(KMeans), 평균 이동(MeanShift), GMM과는 다르게 군집의 모양이나 개수에 대한 사전 가정 없이 군집화가 가능합니다.
 - 예를 들어, 내부 원과 외부 원처럼 복잡한 형태의 데이터 분포에서도 DBSCAN은 유연하게 군집화할 수 있습니다.



✅ DBSCAN 핵심 개념

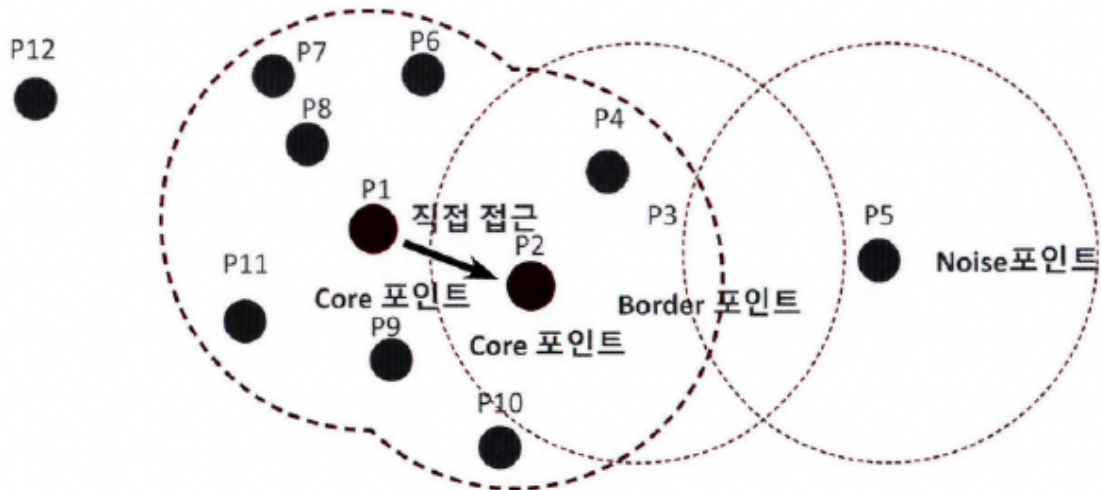
DBSCAN은 다음 두 가지 주요 파라미터를 기반으로 작동합니다:

- epsilon (eps): 중심점 기준 반지름 거리, 즉 주변 이웃 영역의 크기
- min_samples: eps 이내에 존재해야 하는 최소 이웃 데이터 수 (자기 자신 포함)

이에 따라 각 데이터 포인트는 다음 네 가지 유형 중 하나로 분류됩니다:

분류	설명
Core Point (핵심 포인트)	eps 반경 내에 최소 <code>min_samples</code> 이상 데이터가 있는 포인트
Neighbor Point (이웃 포인트)	eps 내에 존재하는 모든 데이터 포인트
Border Point (경계 포인트)	이웃은 충분치 않지만 core point와 이웃인 포인트
Noise Point (잡음 포인트)	핵심 포인트도 아니고, 경계 포인트에도 포함되지 않는 포인트

군집화는 Core Point 간의 직접 연결을 반복하여 확장해 나가는 방식으로 수행됩니다.



✅ 사이킷런 DBSCAN 클래스 사용

- 클래스: `sklearn.cluster.DBSCAN`
- 주요 파라미터:
 - `eps`: 반지름 거리 (보통 1 이하 권장)
 - `min_samples`: core point로 인정받기 위한 최소 데이터 수 (자기 자신 포함)
 - `metric`: 거리 측정 방식 (기본값은 `euclidean`)

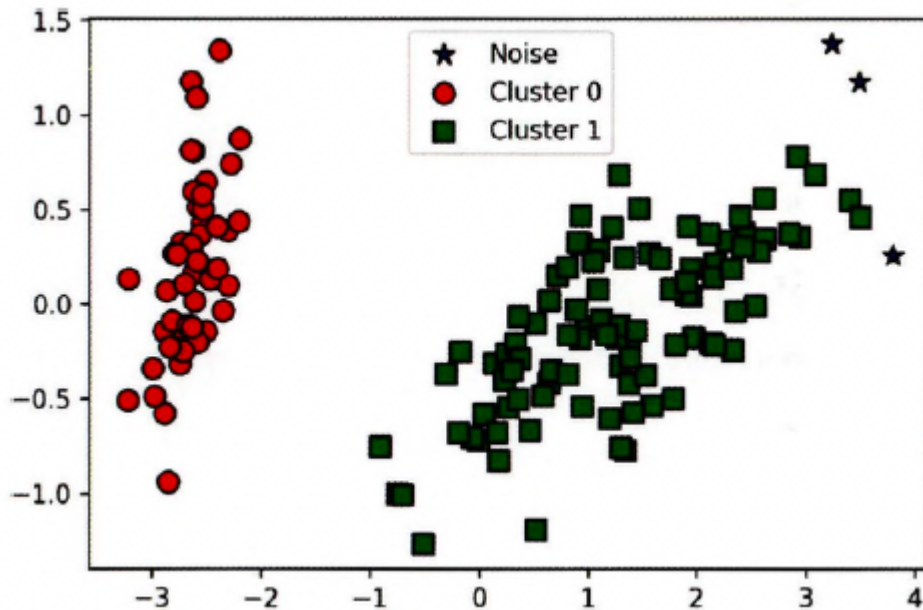
결과 레이블에서 -1은 노이즈를 의미합니다.

✅ 붓꽃 데이터 세트에 DBSCAN 적용

- `eps=0.6`, `min_samples=8`로 설정하여 DBSCAN 수행
- 결과:
 - 군집 레이블: 0, 1, -1 (노이즈)
 - Target이 3개임에도 2개의 군집으로 나뉨
 - → 이는 DBSCAN이 자동으로 적절한 군집 개수를 찾아준 것

시각화를 위해 PCA를 통해 2차원으로 축소 후 군집 시각화 수행:

노이즈는 별표 등으로 표시됨.



✓ eps와 min_samples 조정 효과

- eps 증가 → 군집 범위 확대 → 노이즈 감소
- min_samples 증가 → 더 많은 이웃 요구 → 노이즈 증가 가능성

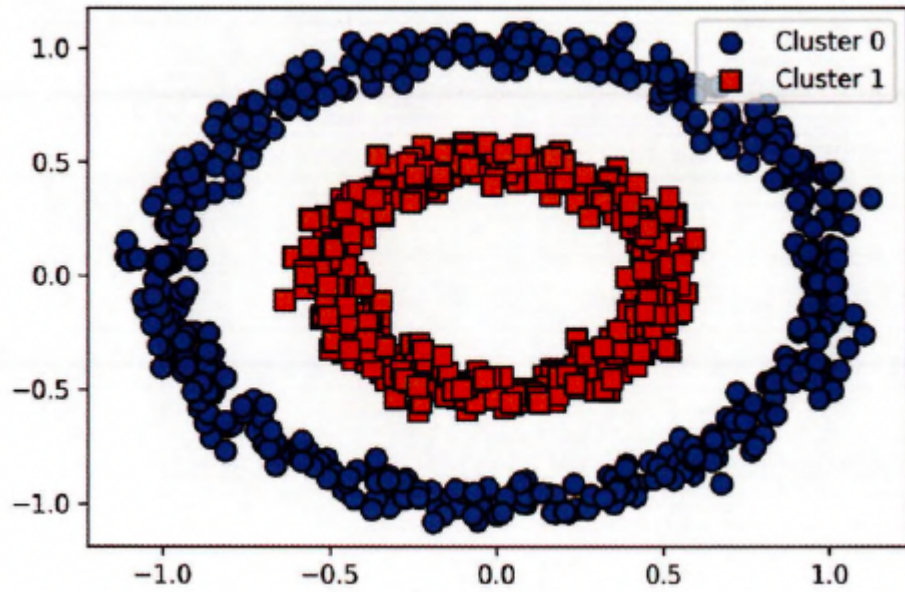
예시:

- eps=0.6 → 노이즈 다수
- eps=0.8 → 노이즈 감소
- min_samples=16 → 노이즈 증가

✓ 복잡한 분포에서 DBSCAN 비교

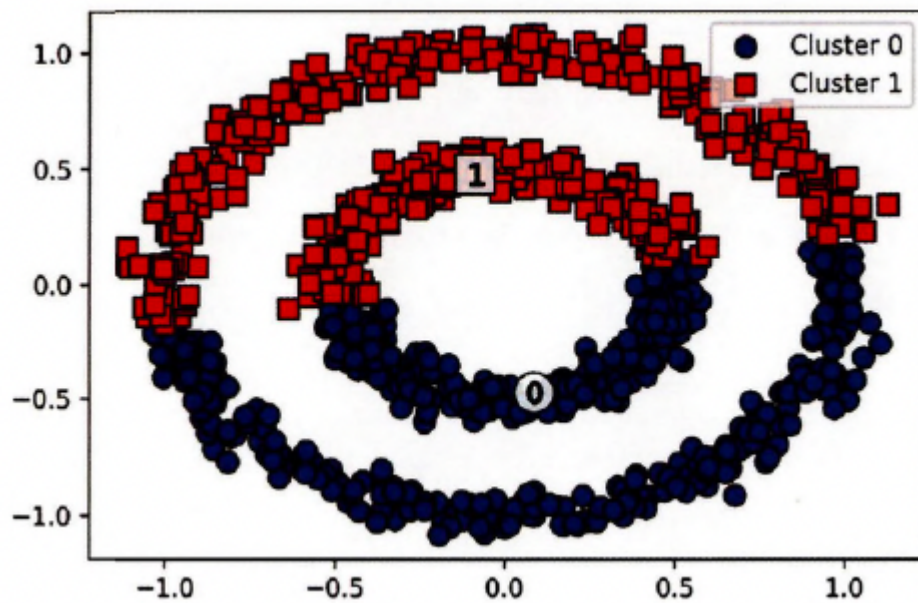
예시: `make_circles()` 함수로 생성한 데이터

- 두 개의 원형 구조 (내부, 외부 원)로 구성된 2차원 데이터



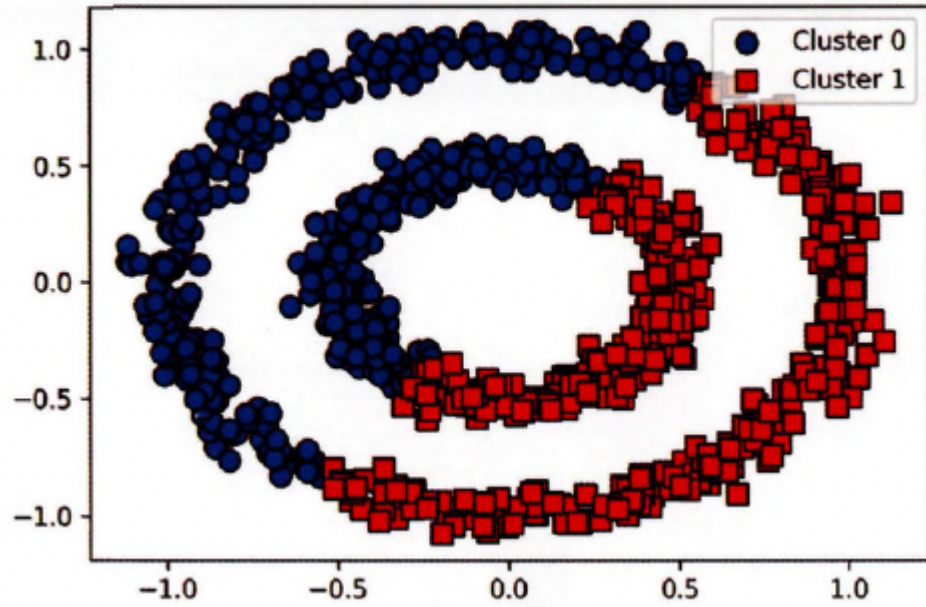
KMeans 적용 결과:

- 중심 기반이기 때문에 원형 구조를 제대로 분리하지 못하고 위/아래로 분리됨



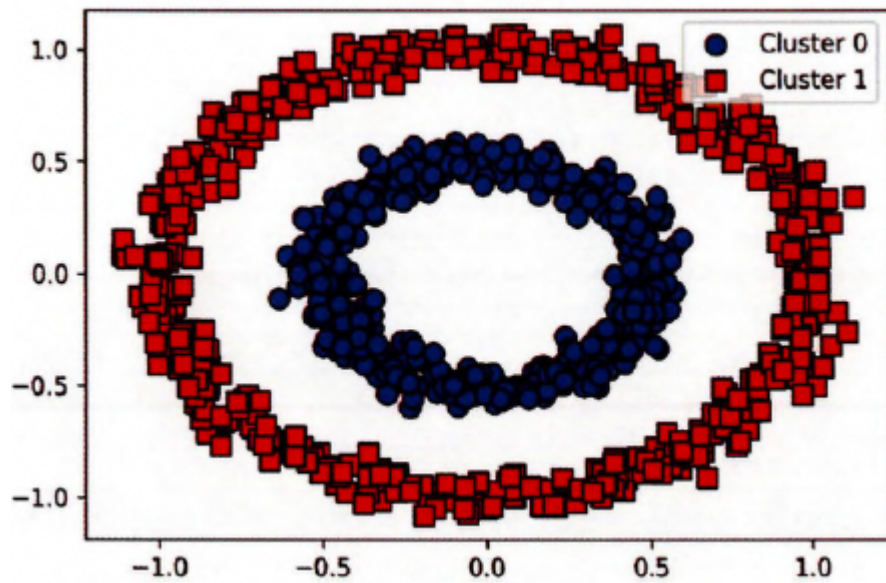
GMM 적용 결과:

- 확률 기반이지만 복잡한 구조에서는 효과 미비



DBSCAN 적용 결과:

- 원형 구조를 정확하게 분리
- 데이터의 형태(밀도, 위치)에 따라 자동으로 군집 형성
- 노이즈도 적절히 식별 가능



✅ 장점, 단점

항목	설명
----	----

장점	- 복잡한 데이터 분포에서도 강력한 군집화- 노이즈 구분 가능- 군집 개수 미리 지정 불필요
단점	- eps, min_samples 설정 민감- 대규모 데이터에서는 속도 이슈
적용 추천	- 기하학적으로 복잡한 분포- 군집 개수 미정의 상황- 이상치 식별이 필요한 경우

06 군집화 실습 - 고객 세그먼테이션

고객 세그먼테이션의 개념

- 정의: 고객 세그먼테이션(Customer Segmentation)은 고객을 다양한 기준으로 분류하여, 맞춤형 마케팅 전략을 수립하는 기법이다. CRM 및 마케팅 전략의 핵심이다.
- 기준 예시: 지역, 성별, 결혼 여부, 직업, 구매주기, 월별 사용액, VIP 상품 사용 여부 등.
- 중요 기준: 실제 판매 중심의 기업에서는 구매 빈도, 금액, 최근 구매 시점이 더 중요하다.

RFM 기법을 통한 고객 분류

- Recency (R): 가장 최근 구매 이후 경과 일수
- Frequency (F): 구매 횟수
- Monetary (M): 총 구매 금액

→ 이 3가지 지표는 고객의 가치를 파악하고 맞춤형 마케팅을 수행하는 데 필수적이다.

데이터 준비 및 전처리

- 데이터셋: UCI의 Online Retail 데이터셋 (541,909건)
- 중요 컬럼: `InvoiceNo`, `StockCode`, `Quantity`, `InvoiceDate`, `UnitPrice`, `CustomerID`, `Country`
- 전처리 작업:
 - `CustomerID` 가 NULL인 데이터 제거
 - `Quantity` 및 `UnitPrice` 가 0 이하인 데이터 제거
 - 영국(`United Kingdom`) 고객만 필터링

→ 최종 데이터 수: 354,321건

RFM 값 생성

- Monetary: `Quantity * UnitPrice`

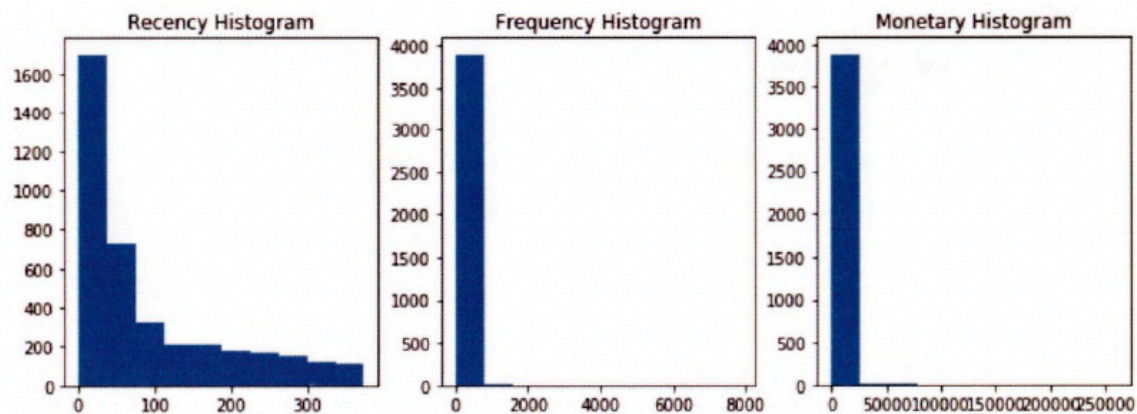
- CustomerID: float → int 변환
- Recency 계산:
 - 기준 날짜: 2011년 12월 10일
 - $\text{Recency} = \text{기준일} - \text{마지막 구매일}$
- GroupBy + Aggregation:

```
aggregations = {
    'InvoiceDate': 'max',
    'InvoiceNo': 'count',
    'sale_amount': 'sum'
}
```

```
cust_df['Recency'] = dt.datetime(2011, 12, 10) - cust_df['Recency']
```

→ 최종 RFM 데이터프레임 생성: **Recency**, **Frequency**, **Monetary**

✅ 데이터 분포 확인 및 왜곡



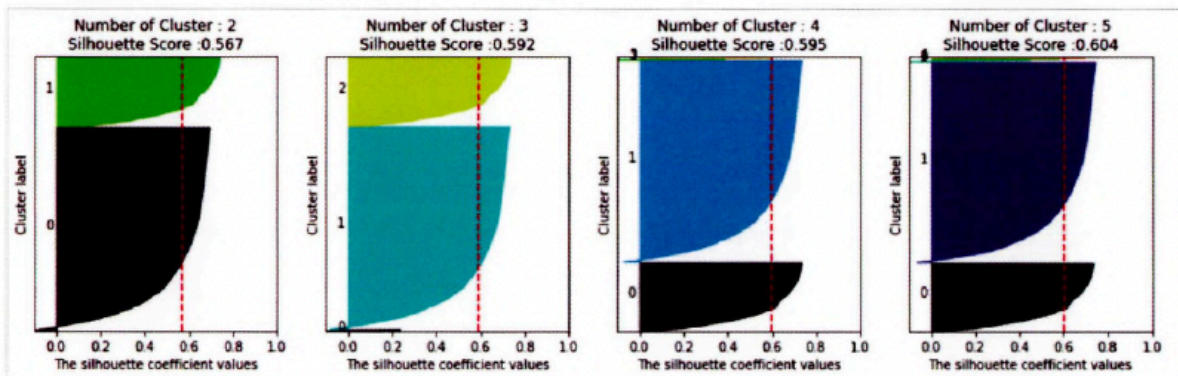
	Recency	Frequency	Monetary
count	3920.000000	3920.000000	3920.000000
mean	92.742092	90.388010	1864.385601
std	99.533485	217.808385	7482.817477
min	1.000000	1.000000	3.750000
25%	18.000000	17.000000	300.280000
50%	51.000000	41.000000	652.280000
75%	143.000000	99.250000	1576.585000
max	374.000000	7847.000000	259657.300000

- `Frequency`, `Monetary` 의 경우 특정 고객의 값이 매우 크며 극단적 분포를 보임
- 히스토그램과 `describe()` 를 통해 왜곡 정도 시각화
- 예시: `Monetary` 의 max값은 259,657.3, 평균은 1864.3, 중앙값은 652.28 → 큰 차이

✓ KMeans 기반 군집화 (로그 변환 전)

- 스케일링: `StandardScaler` 사용
- 군집 수: 3
- 평균 실루엣 스코어: 0.592
- 문제점: 군집이 한쪽으로 쏠리고, 극단값에 의해 왜곡된 군집 분포 발생

[Output]

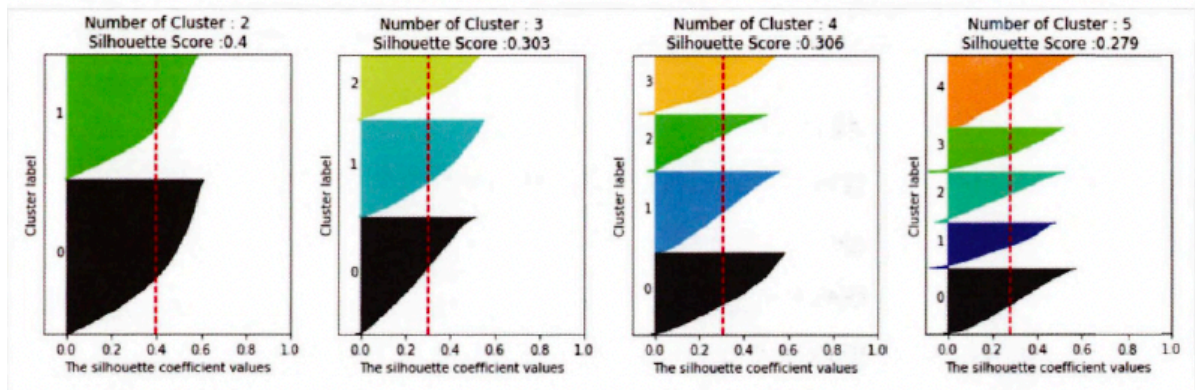


✓ 로그 변환 후 군집화

- 로그 변환 적용: `np.log1p()` 이용해 `Recency`, `Frequency`, `Monetary` 변환

- 재스케일링 및 군집화
- 결과:
 - 실루엣 스코어는 0.303으로 다소 하락
 - 그러나 군집 간 균형이 개선됨

→ 왜곡된 데이터에 로그 변환은 군집의 균형과 분별력을 높이는 데 효과적임



📌 07 정리

✅ K-평균 (K-Means)

- 거리 기반으로 중심점을 이동하며 군집화
- 쉽고 널리 사용되지만, 복잡한 데이터 구조에는 한계
- 군집 개수 선택과 평가가 어려움 → 실루엣 계수 활용

✅ 평균 이동 (Mean Shift)

- 데이터 밀도가 가장 높은 방향으로 중심 이동
- 주로 이미지/영상 분석 등 컴퓨터 비전 분야에서 활용
- GMM (Gaussian Mixture Model)
 - 여러 정규 분포가 섞인 것으로 가정하고 군집화
 - K-평균보다 유연하지만, 시간이 오래 걸리는 단점

✅ DBSCAN

- 밀도 기반 군집화

- 데이터 밀도에 따라 핵심/이웃/경계/잡음 포인트를 구분
 - 복잡한 데이터 구조에도 잘 작동하며, 군집 개수 설정 불필요
-